

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO 1

Implementación de Monolito con patrón MVC, DAO y API
REST en .NET 7

Santiago Mesa
Camilo Peñuela
Sergio Orosman Parra

Docente: Andrés Sánchez

Asignatura: Arquitectura de Software

11 de mayo de 2026
Bogotá D.C., Colombia

Índice

1	Marco Conceptual	2
1.1	Tecnologías Utilizadas	2
1.2	Arquitectura Monolítica	2
1.3	Patrón MVC (Model-View-Controller)	2
1.4	Patrón DAO / Repository	2
2	Diseño del Sistema	3
2.1	Modelo Relacional	3
2.2	Arquitectura de Orquestación (Docker)	4
2.3	Flujo de Interacción (Patrón MVC)	5
3	Procedimiento y Tareas Realizadas	5
3.1	Limpieza y Automatización del Script SQL	5
3.2	Corrección de Enrutamiento (Routing)	6
3.3	Generación de Vistas MVC Faltantes	6
4	Conclusiones	6
5	Referencias	6

1. Marco Conceptual

Este proyecto explora la implementación práctica de patrones de diseño fundamentales en la construcción de aplicaciones empresariales.

1.1. Tecnologías Utilizadas

Para el desarrollo de este taller se integraron las siguientes herramientas y tecnologías de la industria:

- **.NET 7 (ASP.NET Core):** Framework principal utilizado para programar toda la lógica de negocio y exposición web.
- **C#:** Lenguaje de programación empleado para la estructuración de clases, modelos y controladores.
- **Entity Framework Core:** Mapeador Objeto-Relacional (ORM) responsable de abstraer y traducir las consultas SQL hacia la base de datos.
- **SQL Server (Azure SQL Edge):** Motor de base de datos relacional. Se eligió la variante Azure SQL Edge por su soporte nativo en arquitecturas ARM64 (macOS Apple Silicon) mediante contenedores.
- **Docker y Docker Compose:** Tecnología de orquestación empleada para empaquetar, levantar e interconectar la base de datos y la aplicación en un entorno completamente aislado.
- **Swagger (OpenAPI):** Herramienta integrada en ASP.NET (**Swashbuckle**) que documenta interactivamente los endpoints de la API REST generada, facilitando las pruebas.

1.2. Arquitectura Monolítica

La arquitectura monolítica es un enfoque tradicional en el desarrollo de software donde todos los componentes de una aplicación (interfaz de usuario, lógica de negocio y acceso a datos) se integran y despliegan como una única unidad indivisible. En este laboratorio, se construyó un sistema que encapsula el consumo de la base de datos y la exposición web en un único paquete.

1.3. Patrón MVC (Model-View-Controller)

El patrón MVC separa la aplicación en tres componentes lógicos principales:

- **Modelos:** Representan los datos de la aplicación y la lógica de negocio.
- **Vistas:** Interfaz visual con la que interactúa el usuario (Archivos `.cshtml` renderizados en el servidor).
- **Controladores:** Procesan las peticiones del usuario, manipulan el modelo y retornan la vista adecuada.

1.4. Patrón DAO / Repository

Para abstraer el acceso a datos y desacoplar la lógica de Entity Framework de los controladores, se utilizó el patrón de repositorio. Se crearon interfaces genéricas (**IPersonaRepository**, **IEstudioRepository**) que encapsulan las operaciones CRUD y que son inyectadas en los Controladores MVC y los Controladores API REST.

API REST Coexistente: Además del motor tradicional MVC para retornar vistas HTML, este monolito implementa controladores nativos de API REST (decorados con `[ApiController]` y `[Route("api/...")]`) que retornan datos puros en formato JSON, abriendo la puerta para futuras integraciones.

2. Diseño del Sistema

El diseño arquitectónico de este laboratorio está estructurado en tres grandes enfoques: la persistencia de datos (Modelo Relacional), el ecosistema de orquestación (Despliegue) y la interacción lógica (MVC/API).

2.1. Modelo Relacional

La aplicación administra información de personas y sus relaciones académicas y telefónicas. A continuación, se ilustra la relación entre las entidades principales:

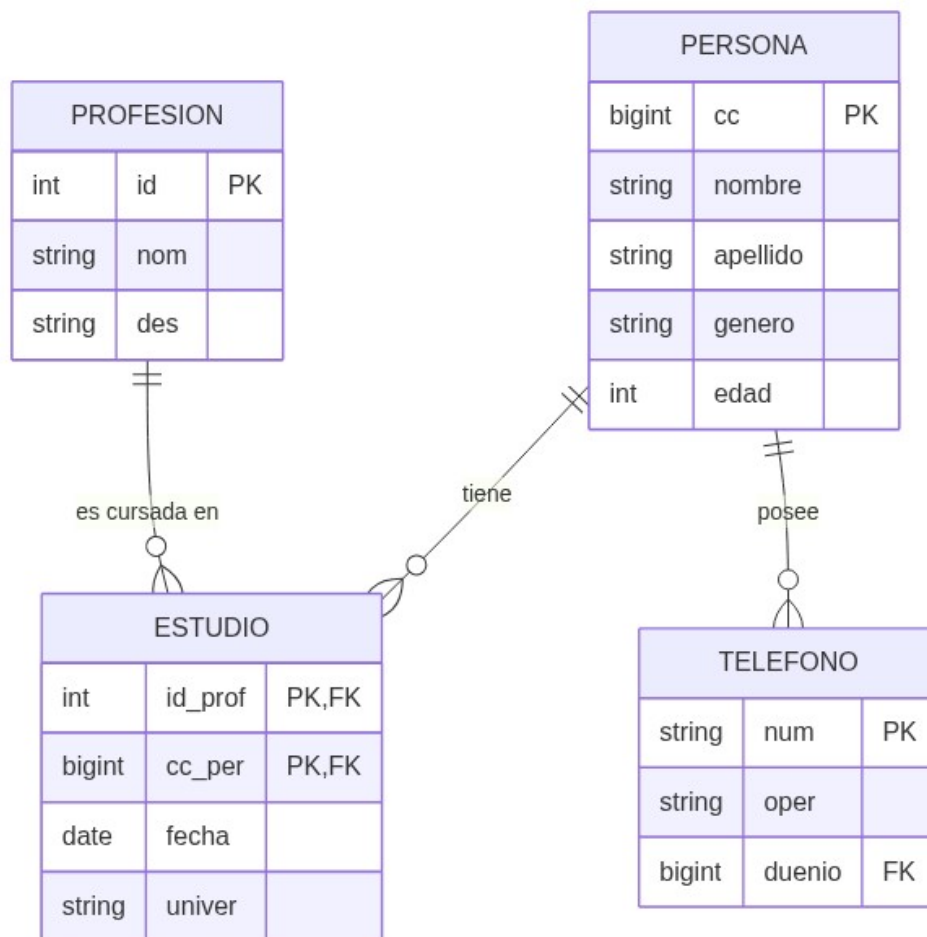


Figura 1: Diagrama Entidad-Relación (Diseñado en Mermaid)

El diagrama de la Figura 1 refleja el esquema base con cuatro tablas clave. **PERSONA** es el eje central identificado por su *cc* (Cédula). La entidad **ESTUDIO** funciona como una tabla puente o asociativa mediante una llave compuesta (*id_prof* y *cc_per*) que une el catálogo de **PROFESION**

con PERSONA. Finalmente, TELEFONO alberga una relación uno a muchos indicando que una persona puede registrar múltiples vías de contacto.

2.2. Arquitectura de Orquestación (Docker)

Para garantizar la correcta ejecución multiplataforma (compatible con macOS y Windows) y solucionar los conflictos típicos de redes y puertos ("localhost"), se encapsuló el ecosistema entero en contenedores.

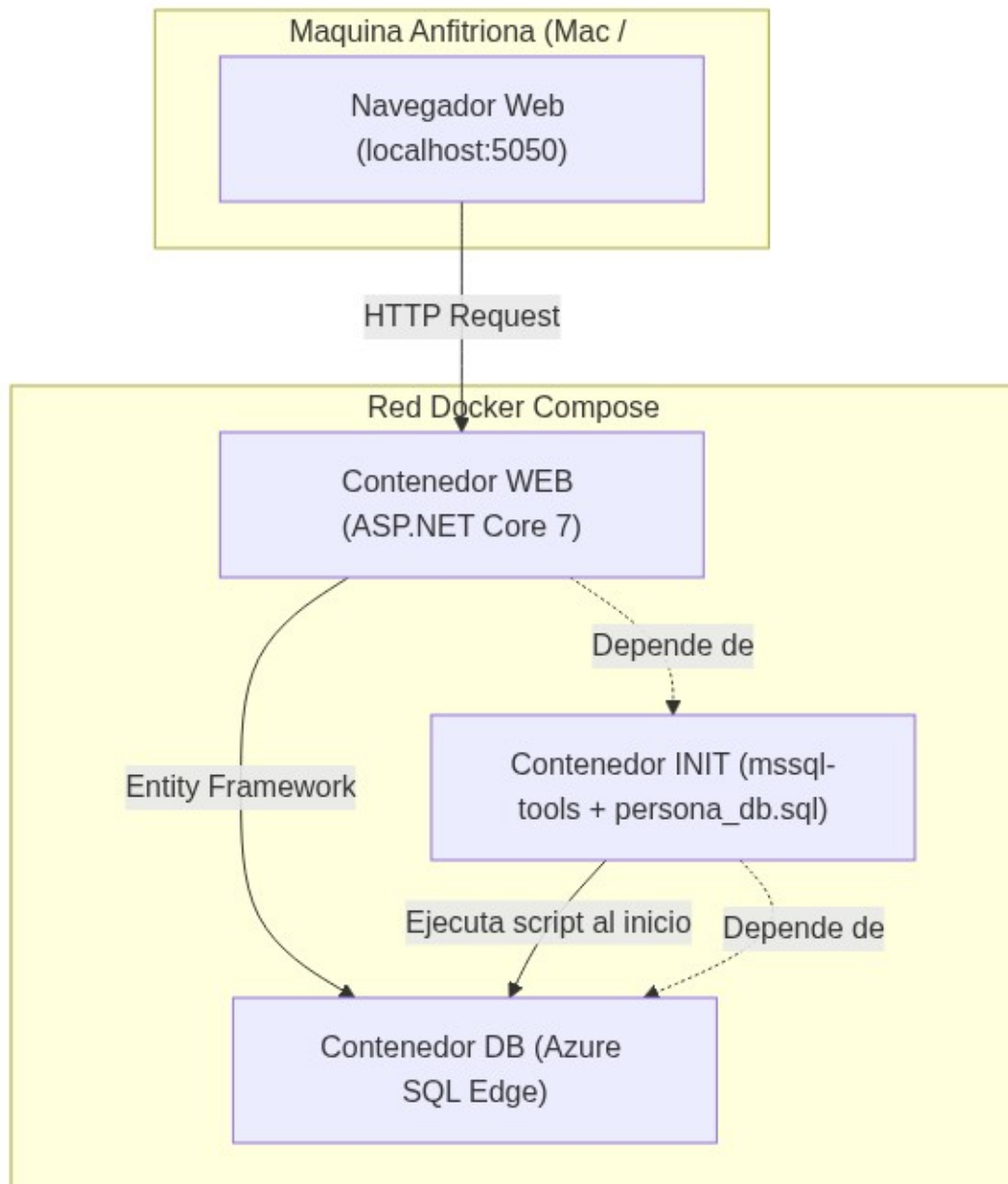


Figura 2: Topología y Arquitectura de Orquestación Docker

Como se observa en la Figura 2, se configuró una red aislada manejada por Docker Compose. Dentro de ella coexisten tres contenedores:

1. **Contenedor DB:** Servidor SQL donde residen los datos físicamente.
2. **Contenedor INIT:** Un contenedor auxiliar ligero cuyo único propósito es esperar a

que la DB despierte, inyectarle el archivo SQL con los datos de prueba masivos de la universidad, y posteriormente apagarse.

3. **Contenedor WEB:** Es nuestra aplicación ASP.NET. Se conecta a la base de datos de manera interna a través de Entity Framework, y expone únicamente el puerto 80 (mapeado al 5050 en el anfitrión) para que el profesor/usuario pueda acceder mediante el navegador.

2.3. Flujo de Interacción (Patrón MVC)

El monolito se encarga de recibir, procesar y retornar la información visual al usuario mediante una estricta separación de responsabilidades.

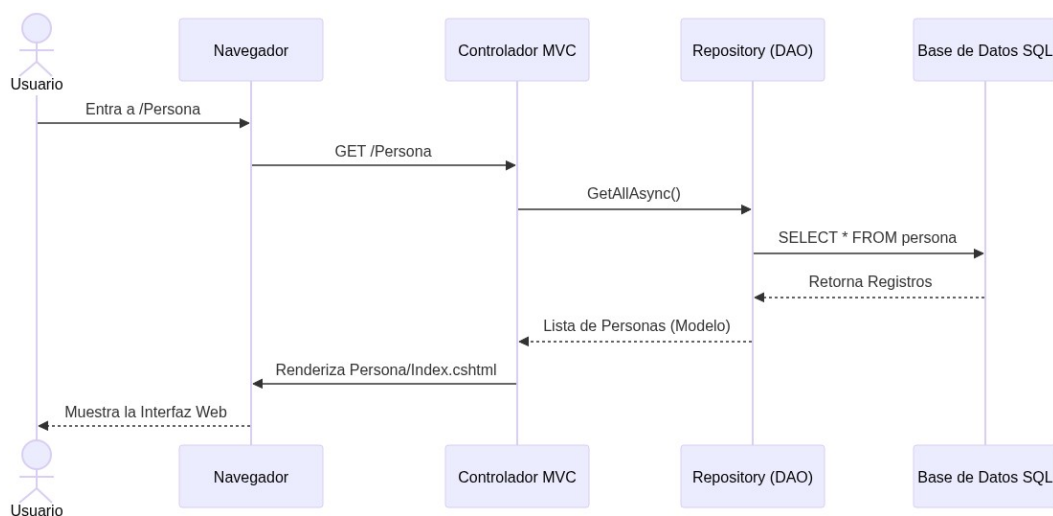


Figura 3: Diagrama de Secuencia: Flujo de Petición MVC

En la Figura 3 se detalla el ciclo de vida de una petición web en la arquitectura construida:

1. El **Usuario** accede mediante su navegador a una ruta como `/Persona`.
2. El enrutador de ASP.NET identifica que se solicita el Controlador `PersonaController` en su acción `Index`.
3. El **Controlador** consume el **Repositorio** (Capa DAO) para solicitar la información.
4. El Repositorio ejecuta una consulta LINQ que se transforma en SQL y consulta directamente a la **Base de Datos**.
5. Al recibir la lista de personas, el Repositorio la mapea en Modelos de C# y se los devuelve al Controlador.
6. El Controlador junta estos Modelos con el archivo de vista correspondiente (`Index.cshtml`) y le devuelve al navegador el archivo final HTML que el usuario visualiza.

3. Procedimiento y Tareas Realizadas

3.1. Limpieza y Automatización del Script SQL

El script original de semilla presentaba errores de sintaxis y conflictos de caracteres CRLF de Windows que causaban que Docker en Mac fallara al inicializar las tablas (Error 500 en la API). Se reescribió el archivo `persona_db.sql` aplicando limpieza de caracteres y agregando instrucciones explícitas de término (GO y QUIT) para automatizar el llenado masivo sin interacción manual.

3.2. Corrección de Enrutamiento (Routing)

Originalmente, el proyecto únicamente resolvía rutas para la API REST debido a la instrucción `app.MapControllers()`. Se modificó `Program.cs` para habilitar el motor MVC tradicional:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

3.3. Generación de Vistas MVC Faltantes

Para cumplir completamente con el patrón MVC, se crearon manualmente 14 archivos `.cshtml` agrupados por controlador. Al crear directorios rigurosos como `Views/Estudio` o `Views/Telefono` se solucionaron múltiples errores de renderizado 500. Ahora, cada tabla cuenta con su respectivo listado (*Index*), creación (*Create*), edición (*Edit*), borrado (*Delete*) y detalles (*Details*), maquetadas con Bootstrap 5.

4. Conclusiones

- **Aislamiento Efectivo:** Utilizar Docker elimina la fricción entre sistemas operativos (macOS vs Windows). Al empaquetar SQL Server y .NET en contenedores mediante `docker-compose`, se asegura que el entorno sea idéntico para cualquier desarrollador que evalúe el código.
- **Híbrido MVC-API:** Este laboratorio demostró cómo .NET permite coexistir de forma eficiente un backend tipo API REST (`[ApiController]`) para consumir datos crudos, con una aplicación web completa orientada a vistas en el mismo paquete monolítico.
- **Mantenibilidad:** El uso del patrón Repository garantiza que si en el futuro se desea cambiar SQL Server por PostgreSQL o MongoDB, los Controladores MVC y API no requerirán modificaciones profundas en su lógica, ya que consumen interfaces genéricas.

5. Referencias

- Microsoft Documentation. (2023). *Tutorial: Create a web API with ASP.NET Core*.
- Guía del Laboratorio 1 (DotNet) proporcionada por la asignatura.
- Fowler, M. (2015). *Microservices and Monoliths*.